



MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Petcov Nicolai FAF-233

Report

Laboratory Work No. 1

Information Security and Cryptography
Caesar Cipher. Caesar Cipher with Permutation

Checked by:
Zaica M., *university assistant*
DISA, FCIM, UTM

Chişinău – 2025

1 Objective

Study and implementation of the Caesar encryption algorithm and its improved variant with alphabet permutation. Analysis of cryptographic resistance and demonstration of vulnerabilities of each variant.

2 Tasks

1. Implementation of the Caesar algorithm for the English alphabet
2. Implementation of the Caesar algorithm with permutation (2 keys)
3. Testing and validation of implementations
4. Exchange of encrypted messages between colleagues
5. Security analysis of implemented algorithms

3 Theoretical Background

3.1 Caesar Cipher

The Caesar cipher is one of the oldest and simplest substitution encryption techniques. In this cipher, each letter of the plaintext is replaced by a letter located at a fixed distance in the alphabet.

Mathematical formulas:

Encryption: $c = e_k(x) = (x + k) \bmod 26$

Decryption: $m = d_k(y) = (y - k) \bmod 26$

where:

- x - numeric value of the character from plaintext
- y - numeric value of the character from ciphertext
- k - secret key ($k \in \{1, 2, 3, \dots, 25\}$)
- $n = 26$ - alphabet length

Example: For $k = 3$ and message CIFRUL CEZAR:

- Preprocessed text: CIFRULCEZAR
- Ciphertext: FLIUXOFHCDU

3.2 Vulnerabilities of the Caesar Cipher

The Caesar cipher is extremely vulnerable to attacks due to its small keyspace:

- Only 25 possible keys exist for the Latin alphabet
- An attacker can try all keys (brute force) in seconds
- Vulnerable to frequency analysis for longer texts

3.3 Caesar Cipher with Permutation

To increase cryptographic resistance, alphabet permutation is applied using a keyword.

Permuted alphabet creation process:

1. Choose a keyword k_2 (minimum 7 characters)
2. Place unique letters from k_2 at the beginning
3. Complete with remaining alphabet letters in order

Example: For $k_2 = \text{cryptography}$:

Normal alphabet: ABCDEFGHIJKLMNOPQRSTUVWXYZ

Permuted alphabet: CRYPTOGAHBDEFIJKLMNOPQSUUVWXZ

Keyspace:

- Simple Caesar cipher: 25 keys
- Caesar cipher with permutation: $26! \times 25 \approx 10^{28}$ keys

Although the keyspace increases dramatically, the algorithm remains vulnerable to frequency analysis because it preserves the statistical distribution of letters.

4 Implementation

4.1 Program Structure

The program was implemented in Python 3 and contains the following components:

- Character-to-number conversion functions
- Input validation functions
- Basic Caesar cipher implementation
- Caesar cipher with permutation implementation
- Brute force attack function
- Interactive user interface

4.2 Auxiliary Functions

```
1 def char_to_num(char):
2     """Convert character to number (A=0, B=1, ..., Z=25)"""
3     return ord(char.upper()) - ord('A')
4
5 def num_to_char(num):
6     """Convert number to character (0=A, 1=B, ..., 25=Z)"""
7     return chr(num + ord('A'))
8
9 def validate_key(key):
10    """Validate key is between 1 and 25"""
11    if not 1 <= key <= 25:
```

```
12         raise ValueError("Key must be between 1 and 25 inclusive")
13     return key
14
15 def preprocess_text(text):
16     """Convert to uppercase and remove spaces"""
17     return ''.join(text.upper().split())
```

Listing 1: Conversion and validation functions

4.3 Caesar Cipher - Task 1.1

```
1 def caesar_encrypt(plaintext, key):
2     """Encrypt plaintext using Caesar cipher"""
3     key = validate_key(key)
4     plaintext = preprocess_text(plaintext)
5
6     ciphertext = ""
7     for char in plaintext:
8         x = char_to_num(char)
9         c = (x + key) % 26
10        ciphertext += num_to_char(c)
11
12    return ciphertext
13
14 def caesar_decrypt(ciphertext, key):
15     """Decrypt ciphertext using Caesar cipher"""
16     key = validate_key(key)
17     ciphertext = preprocess_text(ciphertext)
18
19    plaintext = ""
20    for char in ciphertext:
21        y = char_to_num(char)
22        m = (y - key) % 26
23        plaintext += num_to_char(m)
24
25    return plaintext
```

Listing 2: Caesar cipher implementation

4.4 Caesar Cipher with Permutation - Task 1.2

```
1 def create_permuted_alphabet(keyword):
2     """Create permuted alphabet from keyword"""
3     keyword = keyword.upper()
4     permuted = ""
5     seen = set()
6
7     # Add unique letters from keyword
8     for char in keyword:
9         if char not in seen:
10            permuted += char
11            seen.add(char)
12
13    # Add remaining letters of alphabet
14    for i in range(26):
15        char = num_to_char(i)
```

```
16         if char not in seen:
17             permuted += char
18
19     return permuted
```

Listing 3: Permuted alphabet creation

```
1 def caesar_permutation_encrypt(plaintext, key1, key2):
2     """Encrypt using Caesar cipher with permuted alphabet"""
3     key1 = validate_key(key1)
4     key2 = validate_keyword(key2)
5     plaintext = preprocess_text(plaintext)
6
7     permuted_alphabet = create_permuted_alphabet(key2)
8
9     ciphertext = ""
10    for char in plaintext:
11        pos = char_to_num(char)
12        new_pos = (pos + key1) % 26
13        ciphertext += permuted_alphabet[new_pos]
14
15    return ciphertext
```

Listing 4: Encryption with permutation

4.5 Brute Force Attack

```
1 def brute_force_attack(ciphertext):
2     """Perform brute force attack on Caesar cipher"""
3     results = []
4     for key in range(1, 26):
5         decrypted = caesar_decrypt(ciphertext, key)
6         results.append((key, decrypted))
7         print(f"{key:2d} | {decrypted}")
8
9     return results
```

Listing 5: Brute force attack implementation

5 Testing and Results

5.1 Test 1: Basic Caesar Cipher

Input data:

- Message: cifrul cezar
- Key: $k = 3$

Result:

- Preprocessed text: CIFRULCEZAR
- Ciphertext: FLIUXOFHCDU
- Decryption: CIFRULCEZAR

5.2 Test 2: Brute Force Attack

Input data:

- Ciphertext: SILKVWFITVRKKRTB

Result:

The program tested all 25 keys and identified:

- Correct key: $k = 17$
- Decrypted message: BRUTEFORCEATTACK

This test clearly demonstrates the Caesar cipher's vulnerability to exhaustive attacks.

5.3 Test 3: Caesar Cipher with Permutation

Input data:

- Message: HELLO WORLD
- Key 1: $k_1 = 3$
- Key 2: $k_2 = \text{cryptography}$

Result:

- Permuted alphabet: CRYPTOGAHBDEFIJKLMNOPQSUUVWXZ
- Preprocessed text: HELLOWORLD
- Ciphertext: ATEESVSNEP
- Decryption: HELLOWORLD

5.4 Test 4: Task 1.3 - Exchange Between Colleagues

Message sent to colleague:

- Original text: SUCCESS
- Keys: $k_1 = 5$, $k_2 = \text{password}$
- Sent ciphertext: HXFFDHH

Message received from colleague:

- Received ciphertext: XKDDKIK
- Received keys: $k_1 = 7$, $k_2 = \text{security}$
- Decrypted text: MESSAGE

Both messages were successfully decrypted, confirming the correctness of the implementation.

6 Security Analysis

6.1 Simple Caesar Cipher

Advantages:

- Extremely simple to implement
- Computationally fast
- Useful for educational purposes

Disadvantages:

- Extremely small keyspace (25 keys)
- Can be broken in seconds through brute force
- Vulnerable to frequency analysis
- Provides no real security in the modern world

6.2 Caesar Cipher with Permutation

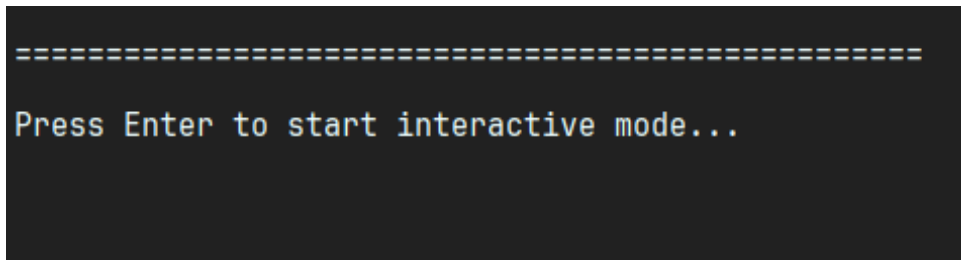
Advantages:

- Dramatically larger keyspace ($\approx 10^{28}$)
- Brute force attack becomes impractical
- Adds an additional layer of complexity

Disadvantages:

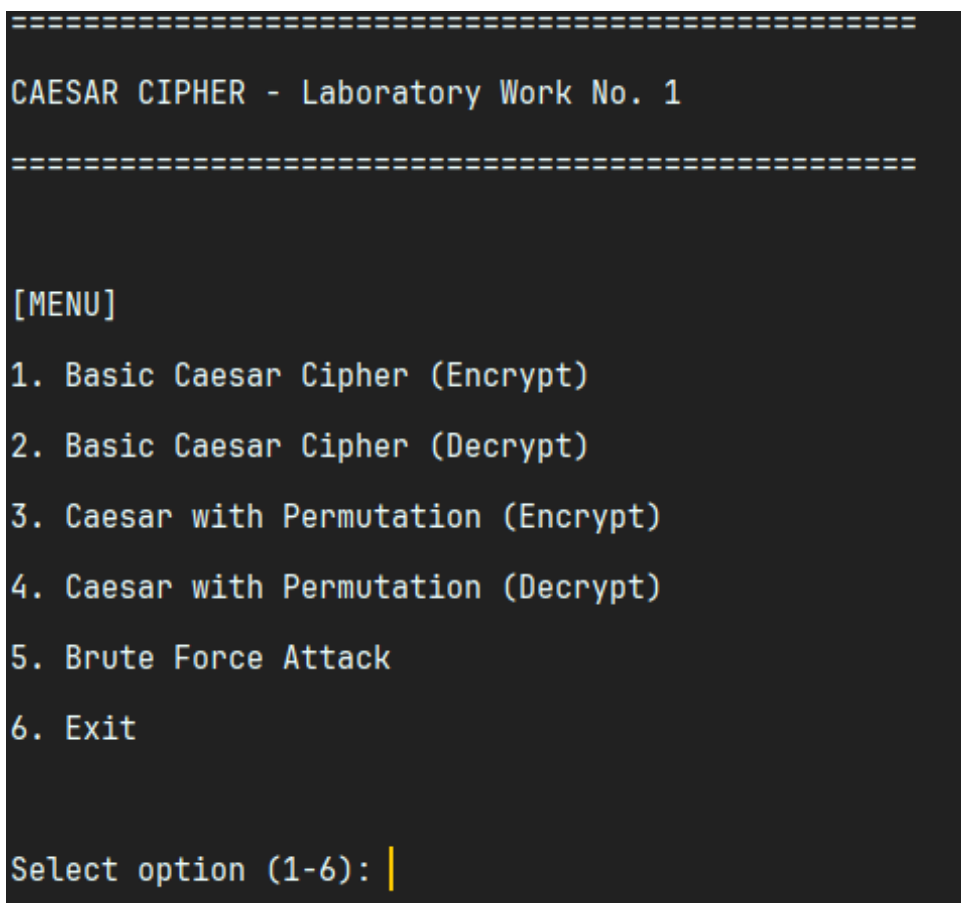
- Still vulnerable to frequency analysis
- Preserves statistical distribution of letters
- Does not resist modern cryptanalytic attacks
- Security depends on keyword secrecy

7 Results



```
=====
Press Enter to start interactive mode...
```

Figure 1: Start



```
=====
CAESAR CIPHER - Laboratory Work No. 1
=====

[MENU]
1. Basic Caesar Cipher (Encrypt)
2. Basic Caesar Cipher (Decrypt)
3. Caesar with Permutation (Encrypt)
4. Caesar with Permutation (Decrypt)
5. Brute Force Attack
6. Exit

Select option (1-6): |
```

Figure 2: Menu


```
Select option (1-6): 1
Enter plaintext: I Love Crypto
Enter key (1-25): 8

Ciphertext: QTWDMKZGXBW

[MENU]
1. Basic Caesar Cipher (Encrypt)
2. Basic Caesar Cipher (Decrypt)
3. Caesar with Permutation (Encrypt)
4. Caesar with Permutation (Decrypt)
5. Brute Force Attack
6. Exit

Select option (1-6): 2
Enter ciphertext: QTWDMKZGXBW
Enter key (1-25): 8

Plaintext: ILOVECRYPTO
```

Figure 3: Problem 1.1

```
Select option (1-6): 3
Enter plaintext: I Love Security
Enter key 1 (1-25): 10
Enter key 2 (keyword, min 7 letters): Cryptology

Permuted alphabet: CRYPTOLGABDEFHIJKMNQSUVWXZ
Ciphertext: NUXOIYIFTRNPA

[MENU]
1. Basic Caesar Cipher (Encrypt)
2. Basic Caesar Cipher (Decrypt)
3. Caesar with Permutation (Encrypt)
4. Caesar with Permutation (Decrypt)
5. Brute Force Attack
6. Exit

Select option (1-6): 4
Enter ciphertext: NUXOIYIFTRNPA
Enter key 1 (1-25): 10
Enter key 2 (keyword, min 7 letters): Cryptology

Permuted alphabet: CRYPTOLGABDEFHIJKMNQSUVWXZ
Plaintext: ILOVESECURITY
```

Figure 4: Problem 1.2

```
Select option (1-6): 3
Enter plaintext: I Love Security
Enter key 1 (1-25): 10
Enter key 2 (keyword, min 7 letters): Crypto

[ERROR] Keyword must have at least 7 characters
Please check the valid range and try again.
```

Figure 5: Problem 1.2

```
Select option (1-6): 1
Enter plaintext: Give me a ten please
Enter key (1-25): 12

Ciphertext: SUHQYQMFQZBXQMEQ
```

Figure 6: Problem 1.3

```
Select option (1-6): 5
Enter ciphertext to attack: SUHQYQMFQZBXQMEQ

Brute force attack on: SUHQYQMFQZBXQMEQ

Key | Decrypted Text
-----|-----
1 | RTGPXPLEPYAWPLDP
2 | QSFOWOKDOXZVOKCO
3 | PRENVNJCWNWYUNJBN
4 | OQDMUMIBMVXTMIAM
5 | NPCLTLHALUWSLHZL
6 | MOBKSKGZKTVRKGYK
7 | LNAJRJFYJSUQJFXJ
8 | KMZIQIEXIRTPIEWI
9 | JLYHPDWHQSOHDVH
10 | IKXGOGCVGPRNGCUG
11 | HJWFNFBUFQMFBTf
12 | GIVEMEATENPLEASE
13 | FHUOLDZSDMOKDZRD
14 | EGTCKCYRCLNJCYQC
15 | DFSBJBXQBKMIBXPB
16 | CERAIAWPAJLHAWOA
17 | BDQZHZVOZIKGZVNZ
18 | ACPYGYUNYHJFYUMY
19 | ZBOXFXTMXGIEXTLX
20 | YANWEWSLWFHDWSKW
21 | XZMVDVRKVEGCVRJV
22 | WYLUCUQJUDFBUQIU
23 | VXKTBTPTCEATPHT
```

Figure 7: Problem 1.3

8 Conclusions

In this laboratory work, the Caesar encryption algorithms were implemented and analyzed in two variants: the classical form and the improved version with alphabet permutation.

Main achievements:

1. Successfully implemented the basic Caesar cipher, respecting all specification requirements
2. Developed the extended version with permutation, significantly increasing the keyspace
3. Practically demonstrated the vulnerability of the simple cipher through brute force attack
4. Completed comprehensive tests, including message exchange between colleagues

Important findings:

- The simple Caesar cipher offers no real security in the modern context
- Alphabet permutation improves resistance to brute force but does not eliminate fundamental vulnerabilities
- Both variants remain susceptible to frequency analysis for longer texts
- Importance of input validation and error handling in cryptographic implementations

Practical applications:

Although these algorithms are not suitable for real security, they remain valuable for:

- Understanding fundamental concepts of cryptography
- Education in the field of information security
- Demonstrating the evolution of cryptographic techniques
- Foundation for understanding modern substitution ciphers

This work demonstrated the importance of combining theoretical analysis with practical implementation for complete understanding of cryptographic algorithms and their limitations.

Source Code

<https://github.com/Nickseen/Cryptology-Security-/tree/Lab1>